

Linear Algebra and Numerical Computing with NumPy

Dr. M. Mallikaarjunan
SASHE, SASTRA University

Vectors: Addition, Scaling, and Dot Product

Simple example

Let

$$u = (2, 3), \quad v = (1, 4).$$

Then

$$u + v = (3, 7), \quad 3u = (6, 9),$$

$$u \cdot v = 2(1) + 3(4) = 14.$$

```
import numpy as np

u = np.array([2, 3])
v = np.array([1, 4])

print(u + v)           # [3 7]
print(3 * u)           # [6 9]
print(np.dot(u, v))    # 14
print(u @ v)           # 14
```

Idea

The dot product produces a scalar and measures directional agreement.

Angle Between Vectors and Cosine Similarity

For nonzero vectors,

$$\cos \theta = \frac{u \cdot v}{\|u\|_2 \|v\|_2}.$$

Take

$$u = (1, 0), \quad v = (1, 1).$$

Then

$$u \cdot v = 1, \quad \|u\| = 1, \quad \|v\| = \sqrt{2},$$

so

$$\cos \theta = \frac{1}{\sqrt{2}} \Rightarrow \theta = 45^\circ.$$

```
u = np.array([1., 0.])
v = np.array([1., 1.])

c = np.dot(u, v) / (
    np.linalg.norm(u) *
    np.linalg.norm(v)
)

theta = np.degrees(np.arccos(c))
print(c)      # 0.7071...
print(theta)  # 45.0
```

Vector Norms

For $x = (3, -4, 12)$:

$$\|x\|_1 = |3| + |-4| + |12| = 19,$$

$$\|x\|_2 = \sqrt{3^2 + (-4)^2 + 12^2} = 13,$$

$$\|x\|_\infty = \max(3, 4, 12) = 12.$$

Since

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}$$

```
import numpy as np
x = np.array([3., -4., 12.])

print(np.linalg.norm(x, 1))      # 19.0
print(np.linalg.norm(x, 2))      # 13.0
print(np.linalg.norm(x))         # 13.0
print(np.linalg.norm(x, np.inf)) # 12.0
```

Outer Product

Let

$$u = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad v = \begin{bmatrix} 4 \\ 5 \end{bmatrix}.$$

Then

$$uv^T = \begin{bmatrix} 1(4) & 1(5) \\ 2(4) & 2(5) \\ 3(4) & 3(5) \end{bmatrix} = \begin{bmatrix} 4 & 5 \\ 8 & 10 \\ 12 & 15 \end{bmatrix}.$$

```
import numpy as np

u = np.array([1, 2, 3])
v = np.array([4, 5])

print(np.outer(u, v))
# [[ 4  5]
#   [ 8 10]
#   [12 15]]
```

Identity, Trace, and Transpose

Let

$$A = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}.$$

Then

$$AI = A, \quad \text{tr}(A) = 2 + 3 = 5, \quad A^T = \begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}.$$

```
A = np.array([[2., 1.],  
              [4., 3.]])  
I = np.eye(2)  
  
print(A @ I)  
print(np.trace(A))  
print(A.T)
```

Determinant: First Test for Invertibility

For

$$A = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix},$$

$$\det(A) = 2(3) - 1(4) = 2 \neq 0.$$

Therefore A is nonsingular and has an inverse.

```
A = np.array([[2., 1.],  
              [4., 3.]])  
  
detA = np.linalg.det(A)  
print(detA)    # 2.0 approximately  
  
if abs(detA) > 1e-10:  
    print("Invertible")
```

Inverse of a 2×2 Matrix

For

$$A = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}, \quad A^{-1} = \frac{1}{2} \begin{bmatrix} 3 & -1 \\ -4 & 2 \end{bmatrix} = \begin{bmatrix} 1.5 & -0.5 \\ -2 & 1 \end{bmatrix}.$$

Check:

$$AA^{-1} = I.$$

```
A_inv = np.linalg.inv(A)
print(A_inv)
print(A @ A_inv)
print(np.allclose(A @ A_inv, np.eye(2)))
```

Best practice

Use `np.linalg.solve(A,b)` for solving $Ax = b$; do not compute $A^{-1}b$ in numerical work.

Solving a Linear System: Hand Method First

Solve

$$2x + y = 13, \quad x + 3y = 19.$$

From the first equation, $y = 13 - 2x$. Substitute:

$$x + 3(13 - 2x) = 19 \Rightarrow -5x = -20 \Rightarrow x = 4.$$

Then

$$y = 13 - 8 = 5.$$

Matrix form:

$$\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 13 \\ 19 \end{bmatrix}.$$

```
A = np.array([[2., 1.], [1., 3.]])  
b = np.array([13., 19.])  
x = np.linalg.solve(A, b)  
print(x)  # [4. 5.]
```

Residual Check

After computing x , calculate

$$r = Ax - b.$$

For $x = (4, 5)^T$,

$$Ax = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \end{bmatrix} = \begin{bmatrix} 13 \\ 19 \end{bmatrix} = b.$$

Hence $r = 0$ and $\|r\|_2 = 0$.

```
x = np.linalg.solve(A, b)
r = A @ x - b
print(r)
print(np.linalg.norm(r))
```

Rank and Singular Systems

Take

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}.$$

Row 2 is $2 \times$ Row 1, so only one row is independent:

$$\text{rank}(A) = 1 < 2.$$

Thus A is singular.

```
A = np.array([[1., 2.],  
              [2., 4.]])  
  
print(np.linalg.det(A))  
print(np.linalg.matrix_rank(A))  
# det = 0, rank = 1
```

Rank Test for $Ax = b$

For n unknowns:

Condition	Conclusion
$\text{rank}(A) = \text{rank}([A b]) = n$	unique solution
$\text{rank}(A) = \text{rank}([A b]) < n$	infinitely many solutions
$\text{rank}(A) < \text{rank}([A b])$	no solution

Simple comparison

With $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$:

- $b = (5, 10)^T$: consistent, infinitely many solutions.
- $b = (5, 7)^T$: inconsistent, no solution.

Eigenvalues and Eigenvectors: A Simple Symmetric Example

Let

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Characteristic equation:

$$\det(A - \lambda I) = (2 - \lambda)^2 - 1 = 0.$$

So

$$\lambda_1 = 3, \quad \lambda_2 = 1.$$

Corresponding directions may be chosen as

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad v_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

```
A = np.array([[2., 1.], [1., 2.]])
vals, vecs = np.linalg.eigh(A)
print(vals)    # [1. 3.]
print(vecs)    #
[[-0.70710678  0.70710678]
 [ 0.70710678  0.70710678]]
```

What Is a Matrix Decomposition?

A decomposition factors a matrix into simpler pieces.

Method	Factorization	Main idea
QR	$A = QR$	orthogonal basis + triangular part
SVD	$A = U\Sigma V^T$	directions + stretching strengths
Cholesky	$A = LL^T$	fast factorization for SPD matrices

Why useful?

Decompositions expose structure, improve numerical stability, and simplify solving, approximation, and data reduction.

QR Decomposition: Hand Example

Take

$$A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = [a_1 \quad a_2] .$$

Two vectors are orthogonal if their dot product is zero:

$$a_1 \cdot a_2 = 1(1) + 1(-1) = 0 .$$

Normalize:

$$q_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} , \quad q_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} .$$

Thus

$$Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} , \quad R = \begin{bmatrix} \sqrt{2} & 0 \\ 0 & \sqrt{2} \end{bmatrix} .$$

Then $A = QR$ and $Q^T Q = I$.

QR Decomposition in NumPy

```
import numpy as np

A = np.array([[1., 1.],
              [1., -1.]])

Q, R = np.linalg.qr(A)

print("Q =\n", Q)
print("R =\n", R)
print("QR =\n", Q @ R)
print("Q^T Q =\n", Q.T @ Q)
print(np.allclose(Q @ R, A))
```

Check

A valid QR factorization must satisfy both $A \approx QR$ and $Q^T Q \approx I$.

SVD: Meaning Before Computation

For any $m \times n$ matrix,

$$A = U\Sigma V^T.$$

Interpretation:

- ① V^T : rotate/re-express the input directions;
- ② Σ : stretch each important direction by a singular value;
- ③ U : rotate/re-express the output directions.

Key fact

The singular values are the square roots of the eigenvalues of $A^T A$.

SVD Step 1–2: Compute $A^T A$ and Singular Values

Use the simple matrix

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 0 \end{bmatrix}.$$

Step 1:

$$A^T A = \begin{bmatrix} 3 & 4 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 4 & 0 \end{bmatrix} = \begin{bmatrix} 25 & 0 \\ 0 & 0 \end{bmatrix}.$$

Step 2: Eigenvalues of $A^T A$ are 25 and 0, hence

$$\sigma_1 = \sqrt{25} = 5, \quad \sigma_2 = 0.$$

So

$$\Sigma = \begin{bmatrix} 5 & 0 \\ 0 & 0 \end{bmatrix}.$$

SVD Step 3–5: Find V , U , and Reconstruct

For $A^T A = \text{diag}(25, 0)$, choose

$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad V = I.$$

For the nonzero singular value,

$$u_1 = \frac{Av_1}{\sigma_1} = \frac{1}{5} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix}.$$

Choose an orthogonal unit vector

$$u_2 = \begin{bmatrix} -4/5 \\ 3/5 \end{bmatrix}.$$

Thus

$$U = \begin{bmatrix} 3/5 & -4/5 \\ 4/5 & 3/5 \end{bmatrix}, \quad A = U\Sigma V^T.$$

SVD in NumPy and Verification

```
import numpy as np

A = np.array([[3., 0.],
              [4., 0.]])

U, S, Vt = np.linalg.svd(A)
Sigma = np.diag(S)
A_back = U @ Sigma @ Vt

print("U =\n", U)
print("S =", S)      # [5. 0.]
print("Vt =\n", Vt)
print("A_back =\n", A_back)
print(np.allclose(A, A_back))
```

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 0 \end{bmatrix} = \begin{bmatrix} -0.6 & -0.8 \\ -0.8 & 0.6 \end{bmatrix} \begin{bmatrix} 5 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} -0.6 & -0.8 \\ -0.8 & 0.6 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 5 & 0 \\ 0 & 0 \end{bmatrix}, \quad V^T = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

Cholesky Decomposition: Hand Calculation

For the symmetric positive definite matrix

$$A = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix},$$

assume

$$L = \begin{bmatrix} \ell_{11} & 0 \\ \ell_{21} & \ell_{22} \end{bmatrix}, \quad A = LL^T.$$

Matching entries:

$$\ell_{11}^2 = 4 \Rightarrow \ell_{11} = 2,$$

$$\ell_{11}\ell_{21} = 2 \Rightarrow \ell_{21} = 1,$$

$$\ell_{21}^2 + \ell_{22}^2 = 3 \Rightarrow \ell_{22} = \sqrt{2}.$$

Hence

$$L = \begin{bmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{bmatrix}.$$

Cholesky in NumPy

```
A = np.array([[4., 2.],  
              [2., 3.]])  
  
L = np.linalg.cholesky(A)  
  
print("L =\n", L)  
print("L L^T =\n", L @ L.T)  
print(np.allclose(L @ L.T, A))
```

Requirement

Cholesky needs a symmetric positive definite matrix.

Least Squares: Hand-Worked Line Fit

Fit $y = mx + c$ to the three points

$$(1, 2), \quad (2, 3), \quad (3, 5).$$

Write

$$A = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}, \quad \theta = \begin{bmatrix} m \\ c \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}.$$

Normal equations:

$$A^T A \theta = A^T b.$$

Here

$$A^T A = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}, \quad A^T b = \begin{bmatrix} 23 \\ 10 \end{bmatrix}.$$

Solving gives

$$m = 1.5, \quad c = \frac{1}{3}.$$

Least Squares with `np.linalg.lstsq`

```
import numpy as np

x = np.array([1., 2., 3.])
y = np.array([2., 3., 5.])

A = np.column_stack((x, np.ones_like(x)))
params, residuals, rank, s = np.linalg.lstsq(
    A, y, rcond=None
)
m, c = params

print(m, c)           # 1.5, 0.3333...
print(A @ params)     # predictions
print(np.mean((y - A @ params)**2))
```

Linear Transformation: 90° Rotation

The counter-clockwise rotation matrix is

$$R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

For $v = (3, 1)^T$,

$$Rv = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}.$$

Thus $(x, y) \mapsto (-y, x)$.

```
R = np.array([[0, -1], [1, 0]])  
v = np.array([3, 1])  
print(R @ v)    # [-1  3]
```

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Numerical Linear Algebra: Core Best Practices

Task	Recommended practice
Solve $Ax = b$	<code>np.linalg.solve(A,b)</code>
Array equality	<code>np.allclose()</code>
Symmetric eigenproblem	<code>np.linalg.eigh()</code>
Overdetermined system	<code>np.linalg.lstsq()</code>
Check factorization	residual norm / <code>allclose</code>
Detect sensitivity	<code>np.linalg.cond(A)</code>

Condition Number

The condition number of a matrix A is defined as:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|$$

Using the 2-norm (Euclidean norm), this simplifies to:

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

where $\sigma_{\max}$ and $\sigma_{\min}$ are the largest and smallest singular values of A , respectively.

$$\kappa(A) = \begin{cases} = 1 & \text{Perfectly conditioned (identity matrix)} \\ < 100 & \text{Well-conditioned (safe to compute)} \\ 100\text{--}1000 & \text{Moderately ill-conditioned (be careful)} \\ > 1000 & \text{Ill-conditioned (results unreliable)} \\ = \infty & \text{Singular matrix (no inverse)} \end{cases}$$

Let

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}.$$

Since A is symmetric, $A^T A = A^2$.

$$A^T A = A^2 = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix}.$$

$$\det(A^T A - \lambda I) = \begin{vmatrix} 10 - \lambda & 6 \\ 6 & 10 - \lambda \end{vmatrix} = (10 - \lambda)^2 - 36 = 0.$$

$$(10 - \lambda)^2 = 36 \quad \Rightarrow \quad 10 - \lambda = \pm 6.$$

$$\lambda_1 = 16, \quad \lambda_2 = 4.$$

$$\sigma_1 = \sqrt{16} = 4, \quad \sigma_2 = \sqrt{4} = 2.$$

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}} = \frac{4}{2} = 2.$$

Well-conditioned. Errors in the input will be amplified by at most a factor of 2.

$$\frac{\|\delta x\|}{\|x\|} \leq 2 \cdot \frac{\|\delta b\|}{\|b\|}.$$

Let

$$B = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}.$$

Since B is symmetric, $B^T B = B^2$.

$$B^T B = B^2 = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}.$$

$$B^T B = \begin{bmatrix} 1(1) + 1(1) & 1(1) + 1(1.0001) \\ 1(1) + 1.0001(1) & 1(1) + 1.0001(1.0001) \end{bmatrix}.$$

$$B^T B = \begin{bmatrix} 2 & 2.0001 \\ 2.0001 & 2.00020001 \end{bmatrix}.$$

Let

$$C = B^T B = \begin{bmatrix} 2 & 2.0001 \\ 2.0001 & 2.00020001 \end{bmatrix}.$$

Trace:

$$\text{tr}(C) = 2 + 2.00020001 = 4.00020001.$$

Determinant:

$$\det(C) = 2(2.00020001) - (2.0001)^2.$$

$$\det(C) = 4.00040002 - 4.00040001 = 0.00000001.$$

The eigenvalues are the roots of:

$$\lambda^2 - (\text{tr})\lambda + (\det) = 0.$$

$$\lambda^2 - 4.00020001\lambda + 0.00000001 = 0.$$

Using the quadratic formula:

$$\lambda = \frac{4.00020001 \pm \sqrt{(4.00020001)^2 - 4(0.00000001)}}{2}.$$

$$\lambda_1 \approx 4.0002, \quad \lambda_2 \approx 0.0000025.$$

$$\sigma_1 = \sqrt{4.0002} \approx 2.00005, \quad \sigma_2 = \sqrt{0.0000025} \approx 0.00158.$$

More precisely, the singular values of B are:

$$\sigma_1 \approx 2.000025, \quad \sigma_2 \approx 0.00005.$$

$$\kappa(B) = \frac{\sigma_{\max}}{\sigma_{\min}} \approx \frac{2.000025}{0.00005} \approx 40,000.$$

$$\boxed{\kappa(B) \approx 40,000}$$

Highly ill-conditioned. A tiny error in the input will be amplified massively.

$$\frac{\|\delta x\|}{\|x\|} \leq 40000 \cdot \frac{\|\delta b\|}{\|b\|}.$$

Vectorization: Why NumPy Is Faster

Python loop:

```
result = [x*x for x in range(1_000_000)]
```

NumPy vectorization:

```
import numpy as np
x = np.arange(1_000_000, dtype=float)
result = x**2
```

Reason

NumPy sends whole-array operations to compiled numerical code, avoiding Python-level work for every element.

Memory Usage and In-Place Operations

For one million values:

```
import numpy as np

a32 = np.zeros(1_000_000, dtype=np.float32)
a64 = np.zeros(1_000_000, dtype=np.float64)

print(a32.nbytes) # 4,000,000 bytes
print(a64.nbytes) # 8,000,000 bytes

values = np.array([1., 2., 3.])
values += 10 # in-place update
```

Tradeoff

float32 saves memory; float64 gives more precision.

Algorithm Choice: Prime Numbers as an Example

Two methods for generating primes up to N :

- repeated trial division: about $O(N\sqrt{N})$;
- Sieve of Eratosthenes: about $O(N \log \log N)$.

Important principle

Choosing a better algorithm usually matters much more than small code-level optimizations.

Numerical Differentiation: Smaller h Is Not Always Better

Forward difference:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

Two competing errors:

- large h : truncation error is large;
- extremely small h : subtraction cancellation and rounding error dominate.

For float64, a useful scale is often near

$$h \approx \sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}.$$

Forward-Difference Experiment

For $f(x) = \sin x$, exact $f'(1) = \cos 1$.

```
import numpy as np

x = 1.0
exact = np.cos(x)

for h in [1e-1, 1e-3, 1e-5,
          1e-7, 1e-9, 1e-11]:
    approx = (np.sin(x+h) - np.sin(x)) / h
    error = abs(approx - exact)
    print(h, error)
```

Expected pattern

The error decreases at first, reaches a minimum region, and then increases again when cancellation becomes dominant.

Performance Profiling with timeit

```
import timeit

t_list = timeit.timeit(
    "[x*x for x in range(1000)]",
    number=10000
)

t_np = timeit.timeit(
    "a**2",
    setup="import numpy as np; a=np.arange(1000)",
    number=10000
)

print(t_list, t_np, t_list/t_np)
```

Practice

Time the same task several times. Avoid drawing conclusions from a single timing run.

Precision and Performance Checklist

- 1 Use `np.isclose()` / `np.allclose()` for floating-point comparison.
- 2 Prefer `float64` for general scientific computing unless memory is critical.
- 3 Check for `NaN` and `Inf` before statistics.
- 4 Avoid unstable formulas and subtraction of nearly equal quantities.
- 5 Prefer `solve` over explicit inverse for linear systems.
- 6 Check the condition number for sensitive problems.
- 7 Prefer vectorized NumPy operations to Python loops.
- 8 Choose a good algorithm before tuning code.
- 9 Verify small examples with known exact answers.

Practice Set 1: Matrices and Linear Algebra

- 1 For $A = \begin{bmatrix} 3 & 1 \\ 5 & 2 \end{bmatrix}$, compute $\det(A)$ and A^{-1} by hand, then verify using NumPy.
- 2 Solve $3x + y = 7$, $x + 2y = 5$ and check the residual.
- 3 Find the rank of $\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$.
- 4 Find the eigenvalues of $\begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}$ and verify $Av = \lambda v$.

Practice Set 2: Decompositions

- 1 Perform QR decomposition of $\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ and verify $Q^T Q = I$.
- 2 Perform SVD of $\begin{bmatrix} 3 & 0 \\ 4 & 0 \end{bmatrix}$ using the five hand steps shown in class.
- 3 Compute the Cholesky factor of $\begin{bmatrix} 9 & 3 \\ 3 & 5 \end{bmatrix}$.
- 4 Fit a least-squares line to $(1, 2), (2, 2), (3, 4)$ and compare hand calculation with `lstsq`.

Practice Set 3: Precision and Performance

- 1 Explain why $0.1 + 0.2 == 0.3$ is false and show the correct comparison.
- 2 Create arrays of one million values in float32 and float64; compare memory usage.
- 3 Compare an $O(n)$ loop with an $O(n^2)$ nested loop using operation counts.
- 4 Use bisection to solve $x^3 - 2 = 0$ on $[1, 2]$ to tolerance 10^{-8} .
- 5 Approximate e^x at $x = 1$ using forward differences for several h values and identify a good range for h .

Complete Workflow for a Matrix Problem

Given a matrix A and right-hand side b :

- ① inspect dimensions and dtype;
- ② compute rank and condition number;
- ③ if square and nonsingular, solve $Ax = b$ with `solve`;
- ④ compute the residual $\|Ax - b\|$;
- ⑤ if symmetric, consider `eigh`;
- ⑥ for decomposition, choose QR / SVD / Cholesky according to the matrix structure;
- ⑦ verify the result numerically with `allclose` or a residual norm.

Question 1: Generator Expression – New Scenario

Scenario: robot vision processing

A mobile robot computes the squared intensity index for 20,000 image samples using a generator:

```
n = 20000
scores = (x*x for x in range(n))
print(scores)
print(sum(scores))
print(sum(scores))
```

- 1 Explain why the second call to `sum(scores)` does not give the same total.
- 2 Rewrite the program so that the total can be obtained twice without storing a large list.
- 3 State what `print(scores)` displays and distinguish a generator expression from a list comprehension.

Question 1: Step-by-Step Solution and Code

- ① `scores` is a **generator object**; values are produced only when requested.
- ② The first `sum(scores)` consumes all values in the generator.
- ③ After exhaustion, the same generator has no remaining values, so the next `sum(scores)` returns 0.
- ④ To obtain the total twice without creating a large list, create a fresh generator each time.

```
n = 20000

def make_scores():
    return (x*x for x in range(n))

scores = make_scores()
print(scores)

s1 = sum(make_scores())
s2 = sum(make_scores())
print(s1)
print(s2)
print("Same total:", s1 == s2)
```

Question 1: Result and Interpretation

Computed result

$$\sum_{x=0}^{19999} x^2 = \frac{19999(20000)(39999)}{6} = 2,666,466,670,000.$$

Therefore both fresh generators produce

$$s_1 = s_2 = 2,666,466,670,000.$$

Typical first line of output

`print(scores)` displays text similar to `<generator object <genexpr> at 0x...>`. The hexadecimal address varies between runs.

A list comprehension, such as `[x*x for x in range(n)]`, creates all values immediately in memory; a generator produces one value at a time.

Question 2: Linear System – New Robotics Scenario

Scenario: robot assembly planning

A robotics factory assembles three modules: base units B , arm units A , and gripper units G . Resource constraints are

$$B + A + G = 60,$$

$$2B + A + 3G = 100,$$

$$B + 4A + 2G = 160.$$

Write a NumPy program to:

- 1 solve for B, A, G ;
- 2 verify the solution by computing MX ;
- 3 compare MX with the original right-hand-side vector r and decide whether the solution is correct.

Question 2: Matrix Form and NumPy Code

Step 1: Write $MX = r$

$$M = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 1 & 3 \\ 1 & 4 & 2 \end{bmatrix}, \quad X = \begin{bmatrix} B \\ A \\ G \end{bmatrix}, \quad r = \begin{bmatrix} 60 \\ 100 \\ 160 \end{bmatrix}.$$

```
import numpy as np

M = np.array([[1, 1, 1],
              [2, 1, 3],
              [1, 4, 2]], dtype=float)
r = np.array([60, 100, 160], dtype=float)

X = np.linalg.solve(M, r)
check = M @ X

print("Solution =", X)
print("M @ X     =", check)
print("r         =", r)
print("Correct? =", np.allclose(check, r))
```


Question 2: Result and Verification

Solution

$$B = 20, \quad A = 30, \quad G = 10.$$

Verification

$$MX = \begin{bmatrix} 20 + 30 + 10 \\ 2(20) + 30 + 3(10) \\ 20 + 4(30) + 2(10) \end{bmatrix} = \begin{bmatrix} 60 \\ 100 \\ 160 \end{bmatrix} = r.$$

Thus `np.allclose(M @ X, r)` returns `True`.

Conclusion

The computed module quantities satisfy all three resource equations, so the numerical solution is correct.

Question 3: 3-D ndarray – New Robotics Scenario

Scenario: energy readings

Store battery-energy readings for **2 robots**, **3 operating modes**, and **4 trials** in a three-dimensional NumPy array.

Use the data

$$R_1 : \begin{bmatrix} 42 & 48 & 45 & 50 \\ 55 & 52 & 58 & 60 \\ 63 & 67 & 65 & 70 \end{bmatrix},$$
$$R_2 : \begin{bmatrix} 38 & 40 & 44 & 46 \\ 49 & 53 & 51 & 55 \\ 59 & 61 & 64 & 66 \end{bmatrix}.$$

Tasks: display the array; report shape, size, and dtype; access one reading by indexing; slice one robot across all modes/trials; and calculate the total for each robot.

Question 3: Step-by-Step NumPy Code

```
import numpy as np

energy = np.array([
    [[42,48,45,50], [55,52,58,60], [63,67,65,70]],
    [[38,40,44,46], [49,53,51,55], [59,61,64,66]]
])

print(energy)                                # (a)
print("Shape:", energy.shape)                # (b)
print("Size :", energy.size)
print("Type :", energy.dtype)

print("R2, mode 3, trial 4:",
      energy[1, 2, 3])                       # (c)

print("All R1 readings:\n",
      energy[0, :, :])                       # (d)

totals = energy.sum(axis=(1, 2))             # (e)
print("Robot totals:", totals)
```

Question 3: Results

Array information

`shape = (2, 3, 4)`, `size = 24`.

Typical integer dtype on a 64-bit system: `int64`.

Slice for Robot 1

$$\begin{bmatrix} 42 & 48 & 45 & 50 \\ 55 & 52 & 58 & 60 \\ 63 & 67 & 65 & 70 \end{bmatrix}$$

Indexed value

`energy[1, 2, 3] = 66`.

Totals

`R1 = 675`, `R2 = 626`.

Question 4: Broadcasting – New Robotics Scenario

Scenario: robot spare-part billing

A robotics service centre records base billing values for four spare-part groups across three service periods:

$$S = \begin{bmatrix} 320 & 360 & 400 \\ 280 & 310 & 350 \\ 450 & 480 & 520 \\ 240 & 270 & 300 \end{bmatrix}.$$

Period-wise discounts are $d = [20, 25, 30]$, and product-group tax rates are

$$t = \begin{bmatrix} 0.05 \\ 0.08 \\ 0.10 \\ 0.06 \end{bmatrix}.$$

Compute the final values after applying discounts and taxes, Show the shape of the result, Compute total revenue by group (sum across periods), Identify the group with maximum revenue, Explain broadcasting and why matrix multiplication is not the right operation for applying tax element-by-element.

Question 4: Vectorized Code and Broadcasting

```
import numpy as np

S = np.array([[320.,360.,400.],
              [280.,310.,350.],
              [450.,480.,520.],
              [240.,270.,300.]])

d = np.array([20., 25., 30.])
t = np.array([[0.05], [0.08], [0.10], [0.06]])

final = (S - d) * (1 + t)
revenue = final.sum(axis=1)
best = np.argmax(revenue)

print("Shape:", final.shape)
print(final)
print("Revenue:", revenue)
print("Maximum group:", best + 1)
```

Broadcasting shapes

$$(4, 3) - (3,) \rightarrow (4, 3), \quad (4, 3) \times (4, 1) \rightarrow (4, 3).$$

Question 4: Computed Result

Final array

315.00	351.75	388.50
280.80	307.80	345.60
473.00	500.50	539.00
233.20	259.70	286.20

Shape: $(4, 3)$.

Revenue by group

$[1055.25, 934.20, 1512.50, 779.10]$.

Hence Group 3 has the maximum revenue, 1512.50 .

Question 4: Why Broadcasting, Not Matrix Multiplication?

- The discount vector has shape $(3,)$, so NumPy repeats it across the four rows when evaluating $S - d$.
- The tax array has shape $(4, 1)$, so each tax rate is repeated across the three columns of its corresponding product group.
- The intended tax calculation is **entrywise**: each billed value is multiplied by its own row's factor $(1 + t_i)$.
- Therefore $*$ is mathematically correct.
- Matrix multiplication $@$ combines rows and columns through dot products. It would represent a different linear transformation and, for these shapes, $(4, 3)@(4, 1)$ is dimensionally invalid because the inner dimensions 3 and 4 do not match.

Conclusion

Use element-wise multiplication with broadcasting for tax/discount adjustments; do not replace it with matrix multiplication.

Question 5: Matrix Inverse – New Robotics Scenario

Scenario: coupled actuator calibration

The coupling matrix for three actuators is

$$C = \begin{bmatrix} 4 & 1 & 2 \\ 1 & 3 & 0 \\ 2 & 0 & 5 \end{bmatrix}.$$

Write a Python program to:

- 1 compute $\det(C)$;
- 2 find C^{-1} ;
- 3 generate I_3 ;
- 4 verify both $CC^{-1} = I$ and $C^{-1}C = I$;
- 5 classify C as singular or nonsingular from its determinant.

Question 5: NumPy Code

```
import numpy as np

C = np.array([[4., 1., 2.],
              [1., 3., 0.],
              [2., 0., 5.]])

detC = np.linalg.det(C)
Cinv = np.linalg.inv(C)
I = np.eye(3)

left  = C @ Cinv
right = Cinv @ C

print("det(C) =", detC)
print("C inverse =\n", Cinv)
print("I =\n", I)
print("C @ Cinv =\n", left)
print("Cinv @ C =\n", right)
print("Verified:",
      np.allclose(left, I) and np.allclose(right, I))
print("Non-singular" if not np.isclose(detC, 0)
      else "Singular")
```

Question 5: Result and Conclusion

Determinant and inverse

$$\det(C) = 43 \neq 0,$$

$$C^{-1} \approx \begin{bmatrix} 0.34884 & -0.11628 & -0.13953 \\ -0.11628 & 0.37209 & 0.04651 \\ -0.13953 & 0.04651 & 0.25581 \end{bmatrix}.$$

Verification

Numerically,

$$CC^{-1} \approx I_3, \quad C^{-1}C \approx I_3,$$

where tiny values of order 10^{-17} may appear instead of exact zeros because of floating-point arithmetic. `np.allclose` therefore returns `True`.

Classification

Since $\det(C) \neq 0$, the actuator-coupling matrix is **nonsingular** and has a unique inverse.

Thank You